

Chapter 1

What is Network Protocol Simulator?

A network protocol simulator is an application where users can perform and simulate tasks related to data communication and network protocols. This includes handling tasks such as bit stuffing and de-stuffing, managing both binary and decimal IPv4 addressing, understanding encoding techniques like NRZ-L and NRZ-I, implementing error detection and correction using Hamming code. This initiative seeks to provide a platform where various network protocol functionalities can be simulated and analyzed. By encompassing these elements, the project aims to offer a practical and educational tool for understanding network protocol behavior. Through the simulation environment, users will be able to explore and experiment with different protocol scenarios, enhancing their understanding of network communication principles. The ultimate goal of the project is to contribute to the development of expertise in network protocol design and implementation.

The motivation behind this project is to make a smart tool which is integrated with multiple network protocol functionalities. This project is about making a tool that helps people understand and improve how computer networks work. By using this application researchers, developers, and network engineers can experiment with different protocols, configurations, and scenarios without the need for physical network infrastructure. It is important because networks are getting more complicated and we need to make sure they can handle all sorts of situations. This simulator will speed up the process of creating a better and more reliable network.

Chapter 2

MECHANISM AND WORKING

2.1 Project Output Screenshots:

2.1.1 Main Menu:

Main menu of the project displays all the available options to the users.

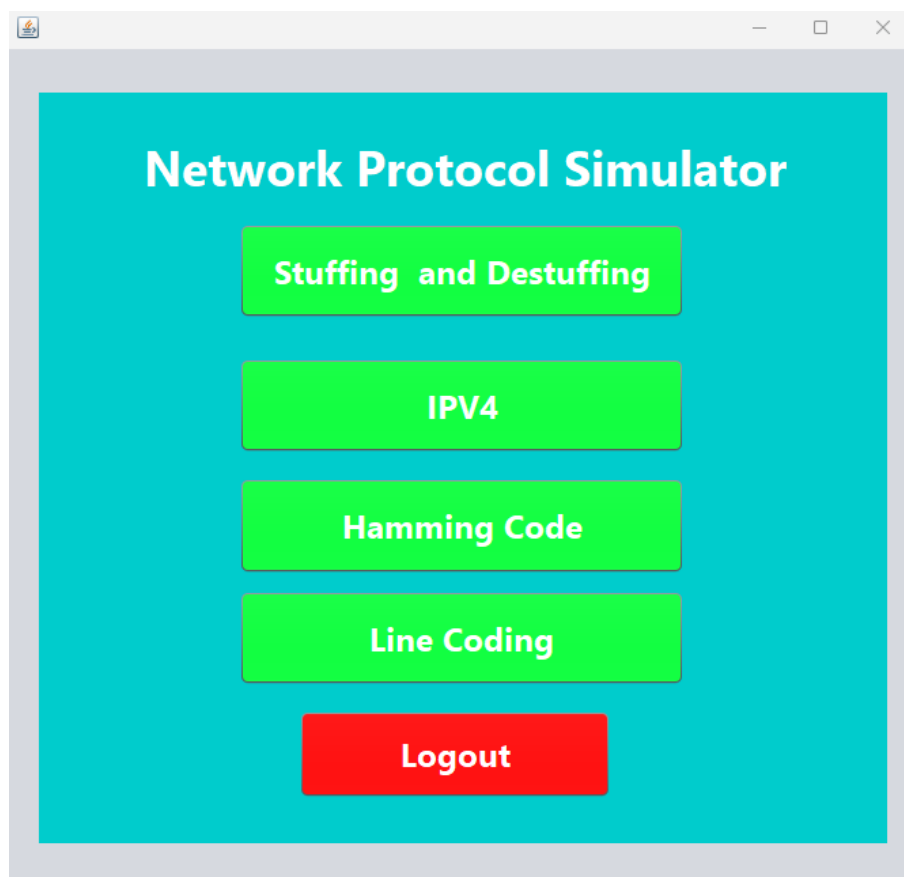


Fig: Main Menu

2.1.2 Stuffing and De-Stuffing:

If the user chooses stuffing De-stuffing then there will be two option

- 1.Character stuffing De-stuffing
- 2.Bit stuffing De-stuffing

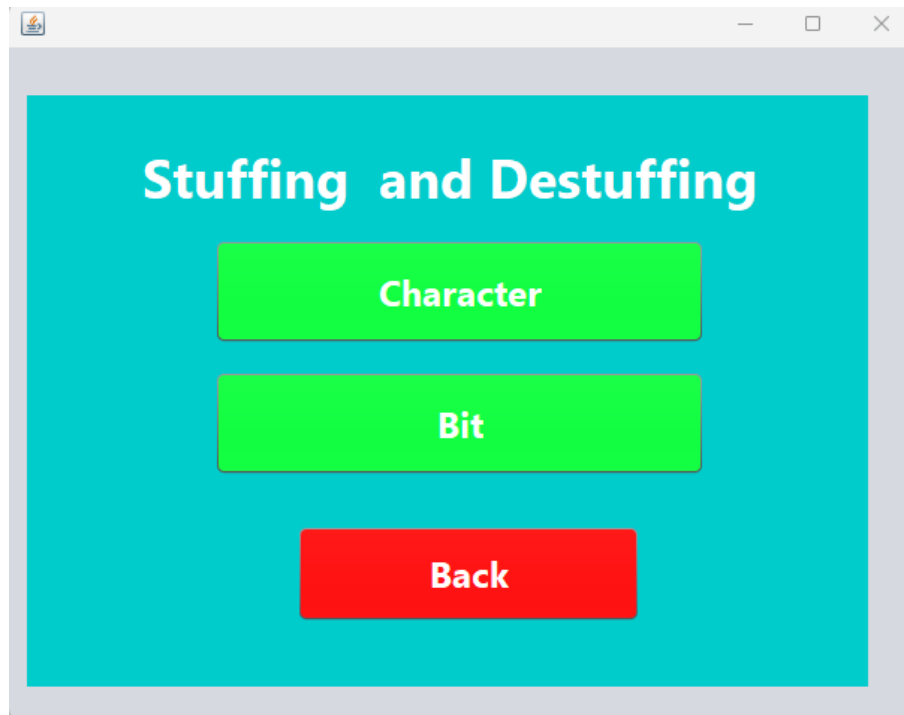


Fig: Stuffing De-stuffing

Choice character Stuffing:

If the user chooses character then an interface will shown for character stuffing. They can also switch to De-stuffing using Options. After that user can give input. And they will get an output.

The screenshot shows a web application window titled "Character Stuffing and Destuffing". The interface has a teal background. On the left, there are labels for "Optin", "FLAG", "INPUT", and "Output". To the right of these labels are input fields. The "Optin" dropdown menu is set to "Stuffing". The "FLAG" field contains the character "a". The "INPUT" field contains the text "Jabed-Fahad". The "Output" field contains the text "aJabed-Fahada". At the bottom of the form, there are three buttons: "Calculate" (green), "Clear" (green), and "Back" (red).

Fig: Character Stuffing

Choice Character De-Stuffing:

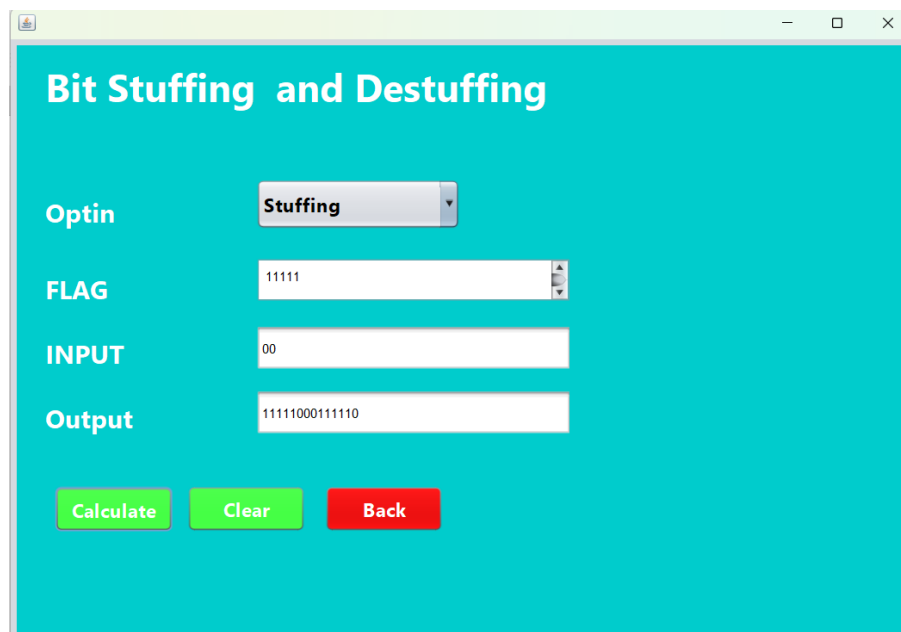
After switching to de-stuffing user can see a new interface for de-stuffing. And user can give input and they will get an output. Users can go back to the menu by clicking the Back button.

The screenshot shows the same web application window, but the "Optin" dropdown menu is now set to "Deshuffling". The "FLAG" field still contains "a". The "INPUT" field now contains the text "aJabed-Fahada", which is the stuffed string from the previous state. The "Output" field now contains the text "Jabed-Fahad", which is the result of de-stuffing. The "Calculate", "Clear", and "Back" buttons remain at the bottom.

Fig: Character De-stuffing

Bit Stuffing

In bit stuffing zero bit is stuffed after five consecutive ones in the input bit stream.

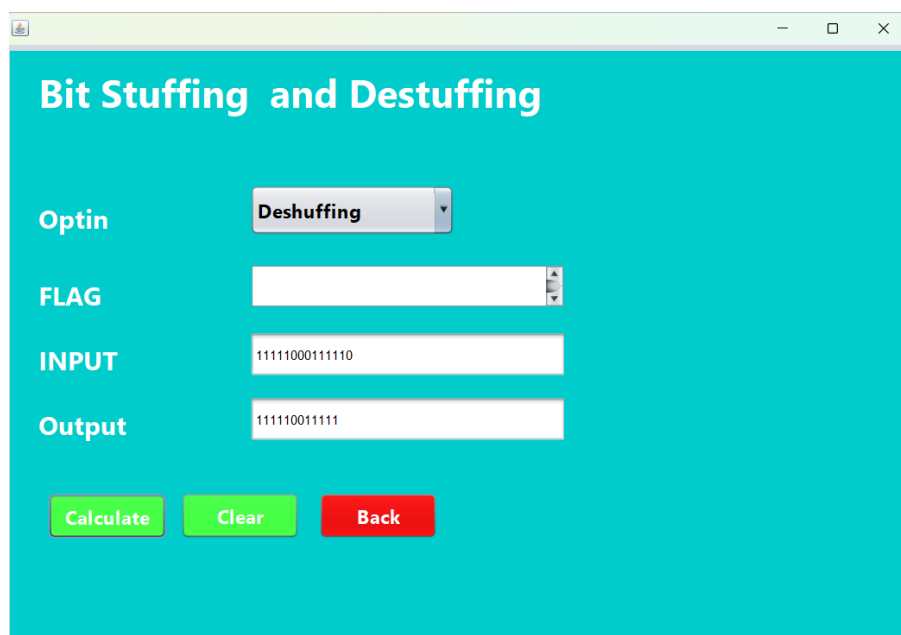


The screenshot shows a web application titled "Bit Stuffing and Destuffing". The "Optin" dropdown menu is set to "Stuffing". The "FLAG" field contains "11111". The "INPUT" field contains "00". The "Output" field displays "11111000111110". At the bottom, there are three buttons: "Calculate" (green), "Clear" (green), and "Back" (red).

Fig: Bit stuffing

Bit de-Stuffing

Bit de-stuffing removes the stuffed bit from the output stream.



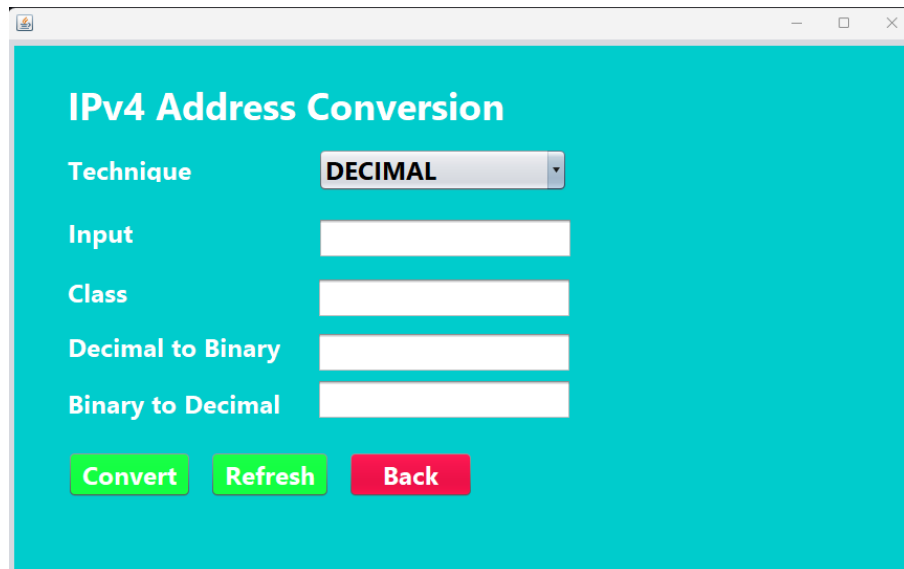
The screenshot shows the same web application, but the "Optin" dropdown menu is now set to "Deshuffing". The "FLAG" field is empty. The "INPUT" field contains "11111000111110". The "Output" field displays "111110011111". The "Calculate", "Clear", and "Back" buttons are still present at the bottom.

Fig: Bit de-stuffing

2.1.3 IPv4:

If the User chooses IPv4 they they can switch to Decimal Conversion Binary Conversion

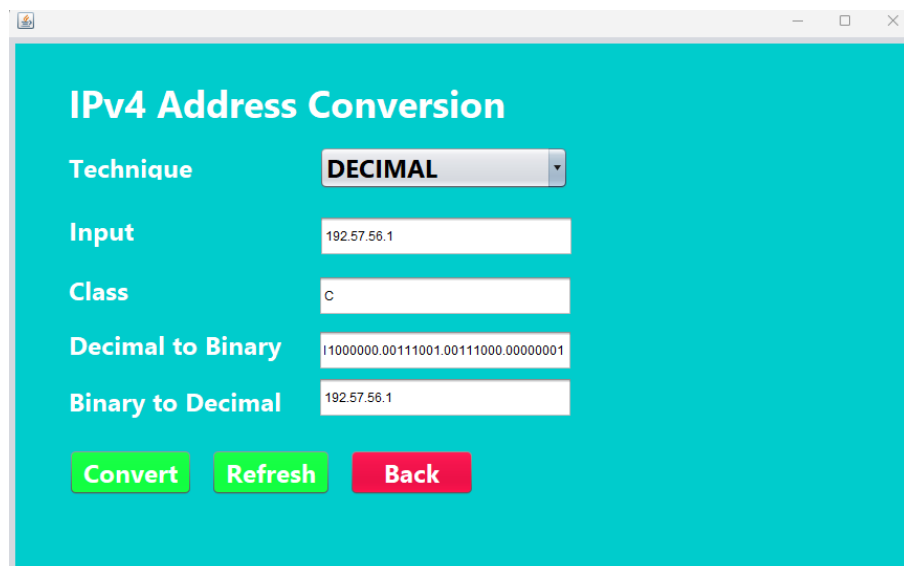
IPv4 Decimal Conversion:



The screenshot shows a web application window titled "IPv4 Address Conversion". It features a teal background. At the top, the title "IPv4 Address Conversion" is displayed in white. Below the title, there is a "Technique" dropdown menu set to "DECIMAL". Underneath, there are five input fields: "Input", "Class", "Decimal to Binary", and "Binary to Decimal". At the bottom, there are three buttons: "Convert" (green), "Refresh" (green), and "Back" (red).

Fig: IPv4 Decimal Conversion

After that users can give input and they will receive a Output.



This screenshot shows the same application after a conversion. The "Input" field now contains "192.57.56.1", the "Class" field contains "C", the "Decimal to Binary" field contains "11000000.00111001.00111000.00000001", and the "Binary to Decimal" field contains "192.57.56.1". The "Technique" dropdown remains set to "DECIMAL". The "Convert", "Refresh", and "Back" buttons are still present at the bottom.

Fig: Decimal to Binary

IPv4 Binary Conversion:

Users Can switch also Binary to Decimal Conversion by Choosing options.

The screenshot shows a web application titled "IPv4 Address Conversion". It features a teal background with white text. The interface includes a dropdown menu for "Technique" set to "BINARY". Below this are input fields for "Input" (containing "11000000.00111001.00111000.00000001"), "Class" (containing "C"), "Decimal to Binary" (containing "192.57.56.1"), and "Binary to Decimal" (containing "11000000.00111001.00111000.00000001"). At the bottom, there are three buttons: "Convert" (green), "Refresh" (green), and "Back" (red).

Fig: Binary to Decimal

2.1.4 Hamming Code:

To perform the hamming code users go back to the main menu and click on the Hamming code, then they can see a user interface from the hamming code.

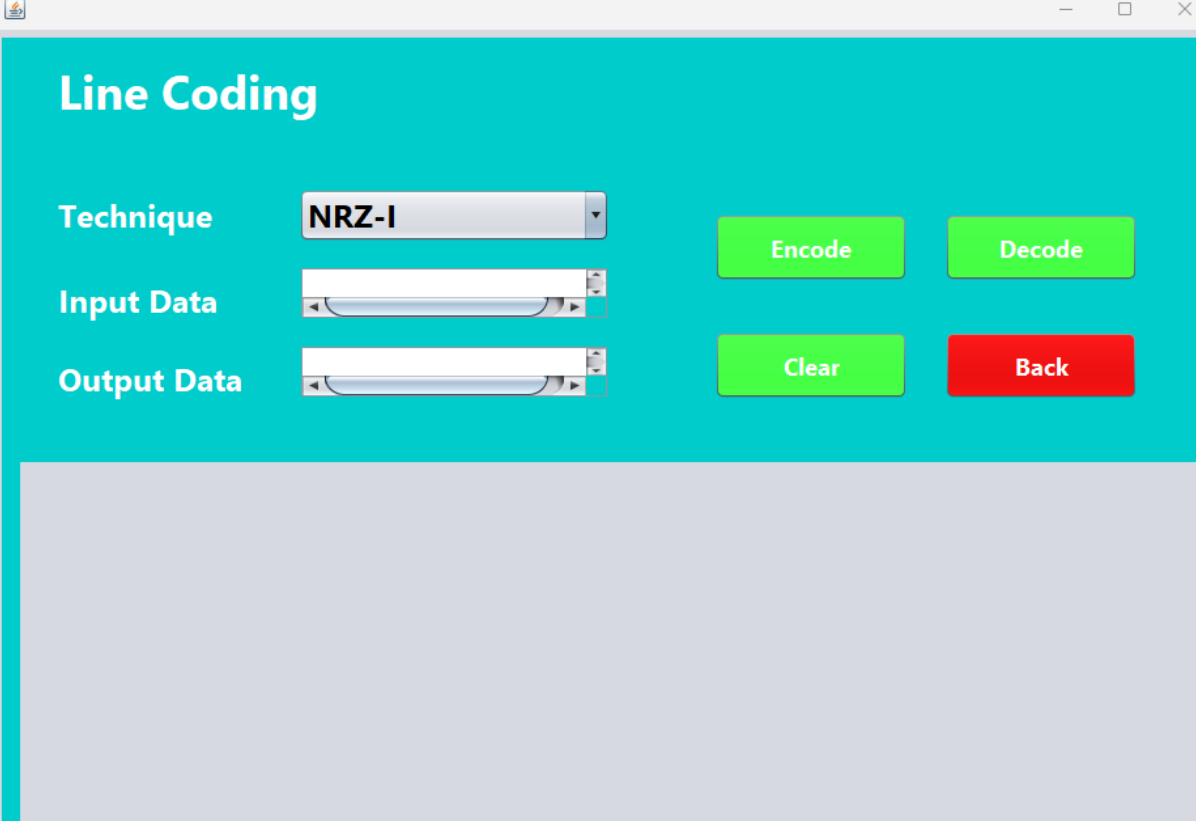
The screenshot shows a web application titled "Hamming Code Error Detection & Correction". It features a teal background with white text. The interface is divided into two columns. The left column contains input fields for "Input Bits" (0111), "Length" (7), "Hamming code" (0001111), and "Parity of Input Bits" (p1=0 p2=0 p4=1). The right column contains input fields for "Rechieved Bits" (0001011), "Error Bit Position" (5), "Corrected Hamming code" (0001111), and "Parity of Rechieved Bits" (p1=1 p2=0 p4=1). At the bottom, there are four buttons: "Hamming Code" (green), "Error Check" (green), "Refresh" (green), and "Back" (red).

Fig: Hamming Code

After that users input the bit for the hamming code and the system will show the output of the hamming length, parity bit, and hamming code. After that user will input the Received bit and click on the Error Check Button it will show the corrected hamming code and parity bits of the error position.

2.2 Line Coding:

To perform the Line Coding users go back to the main menu and click on the Line Coding, then they can see a user interface from the Line Coding. Users can Perform various line coding operations like NRZ-I, NRZ-L, AMI, PSUDU TERNARY, MANCHESTER, DIFFERENTIAL MANCHESTER.



The image shows a web-based user interface for line coding. The interface has a teal header with the title "Line Coding". Below the header, there are three input fields: "Technique" (a dropdown menu currently showing "NRZ-I"), "Input Data" (a text input field), and "Output Data" (a text input field). To the right of these fields are four buttons: "Encode" (green), "Decode" (green), "Clear" (green), and "Back" (red). The interface is displayed within a browser window with standard window controls (minimize, maximize, close) in the top right corner.

Fig: Line Coding

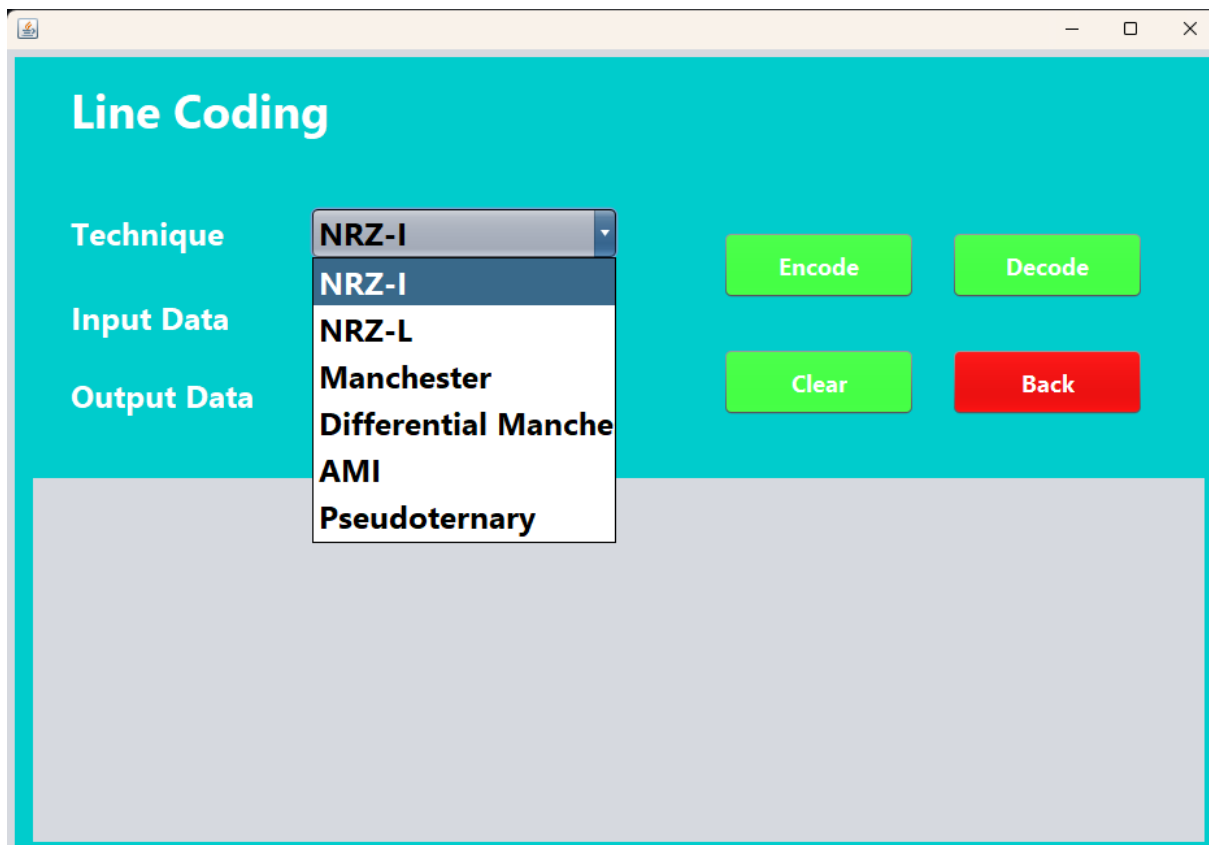


Fig: Line Coding Options

2.2.1 NRZ-I:

After Choosing the option NRZ-I users can see the interface for NRZ-I , After inserting the input they will get a graph of NRZ-I Line Coding. After inserting the Input user click on the Encode button ,then they will see the NRZ-I Graph.

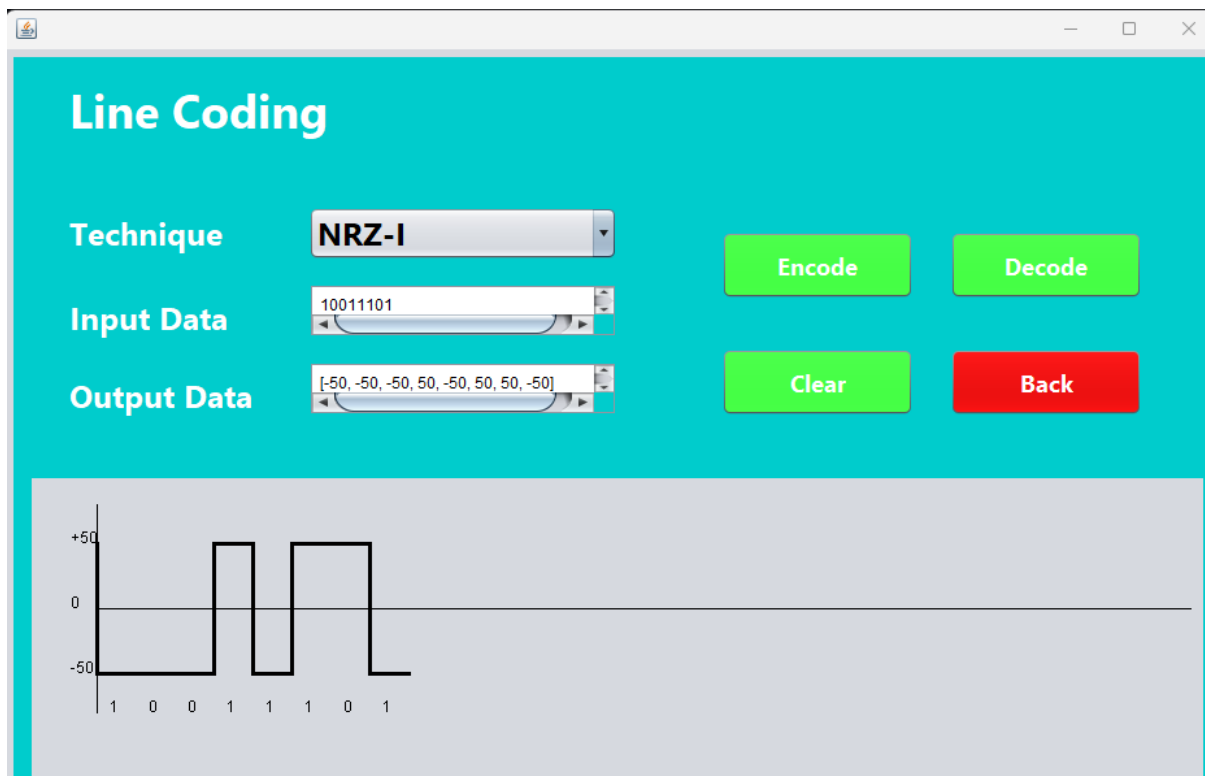


Fig: NRZ-I Encoding

When user click on the Decode button, then they will see the Decoded bits of NRZ-I graph.

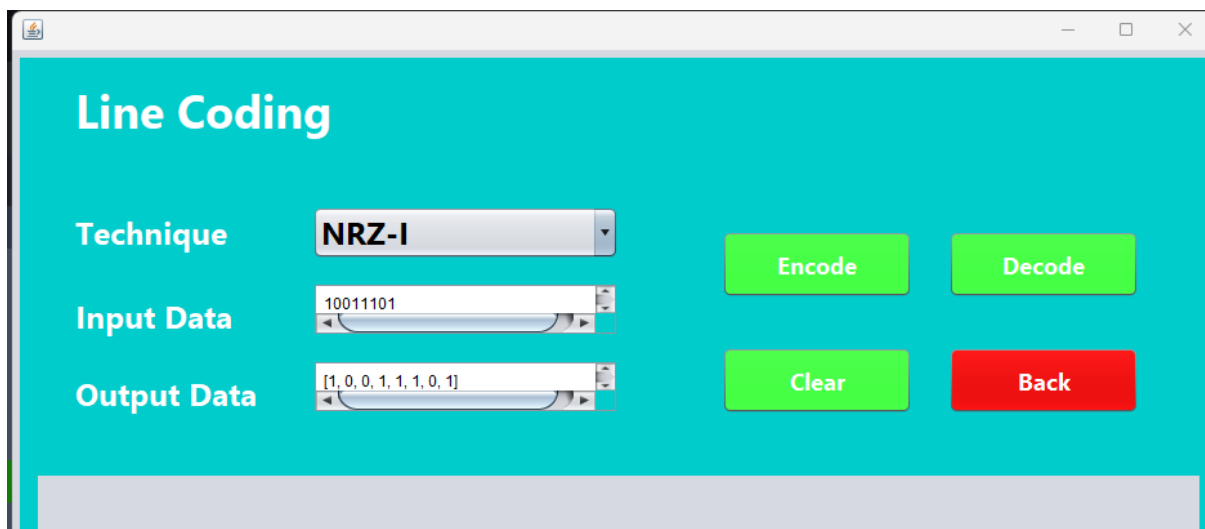


Fig: NRZ-I Decoding

2.2.2 NRZ-L:

After Choosing the option NRZ-L users can see the interface for NRZ-L , After inserting the input they will get a graph of NRZ-I Line Coding. After inserting the Input users clicks on the Encode button, then they will see the NRZ-L Graph.

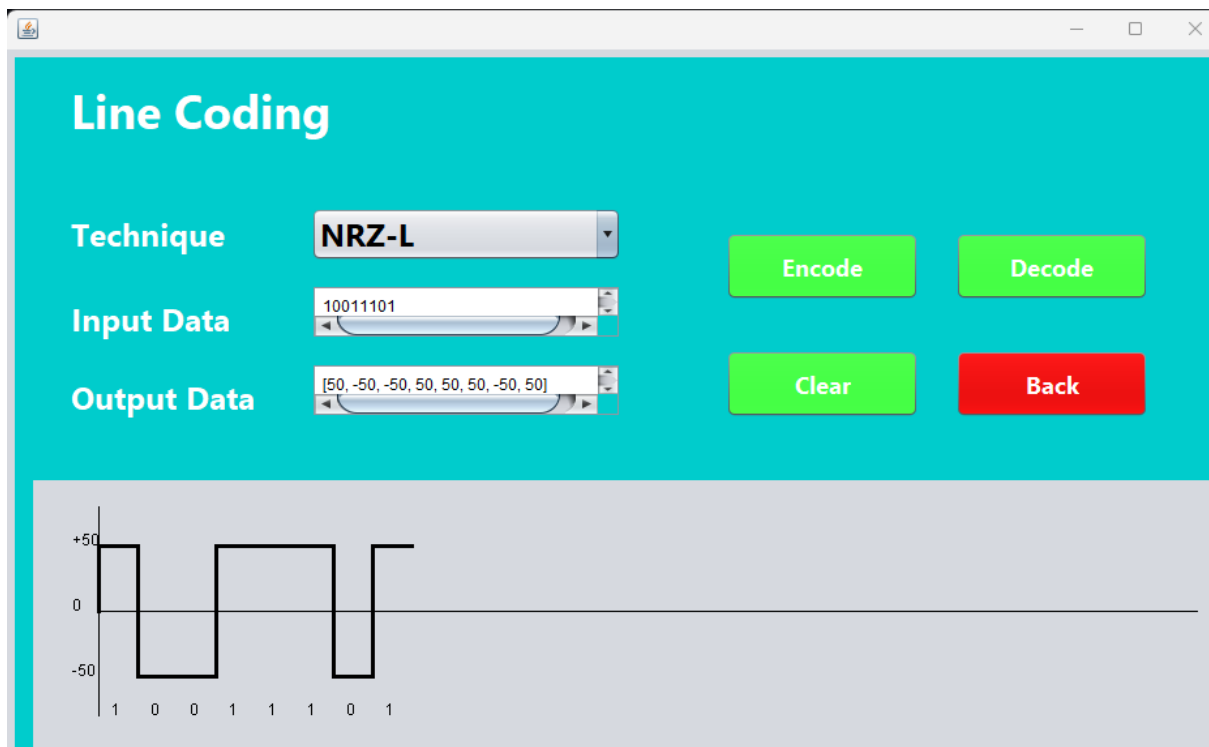


Fig: NRZ-L Encoding

When user click on the Decode button, then they will see the Decoded bits of NRZ-L graph.



Fig: NRZ-L Decoding

2.2.3 MANCHESTER:

After Choosing the option Manchester users can see the interface for Manchester, After inserting the input they will get a graph of Manchester Line Coding. After inserting the Input users click on the Encode button, then they will see the Manchester Graph.

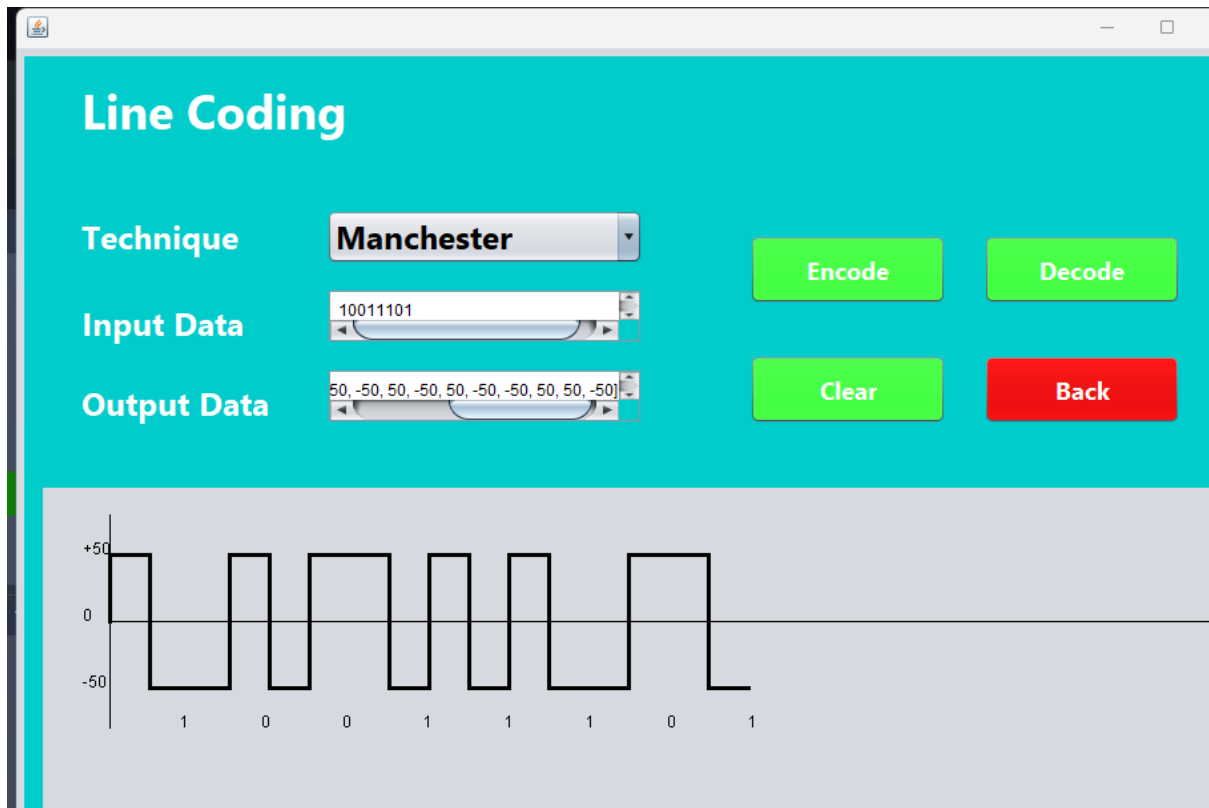


Fig: Manchester Encoding

When users click on the Decode button, then they will see the Decoded bits of the manchester graph.

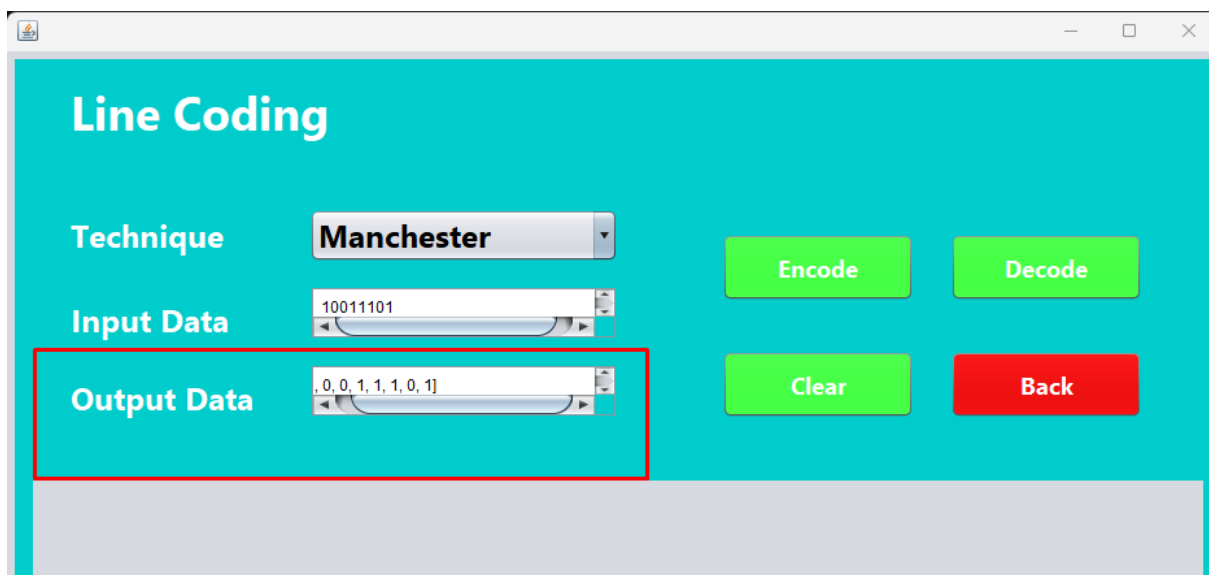


Fig: Manchester Decoding

2.2.4 DIFFERENTIAL MANCHESTER:

After Choosing the option Differential Manchester users can see the interface for Differential Manchester, After inserting the input they will get a graph of Differential Manchester Line

Coding. After inserting the Input users click on the Encode button, then they will see the Differential Manchester Graph.

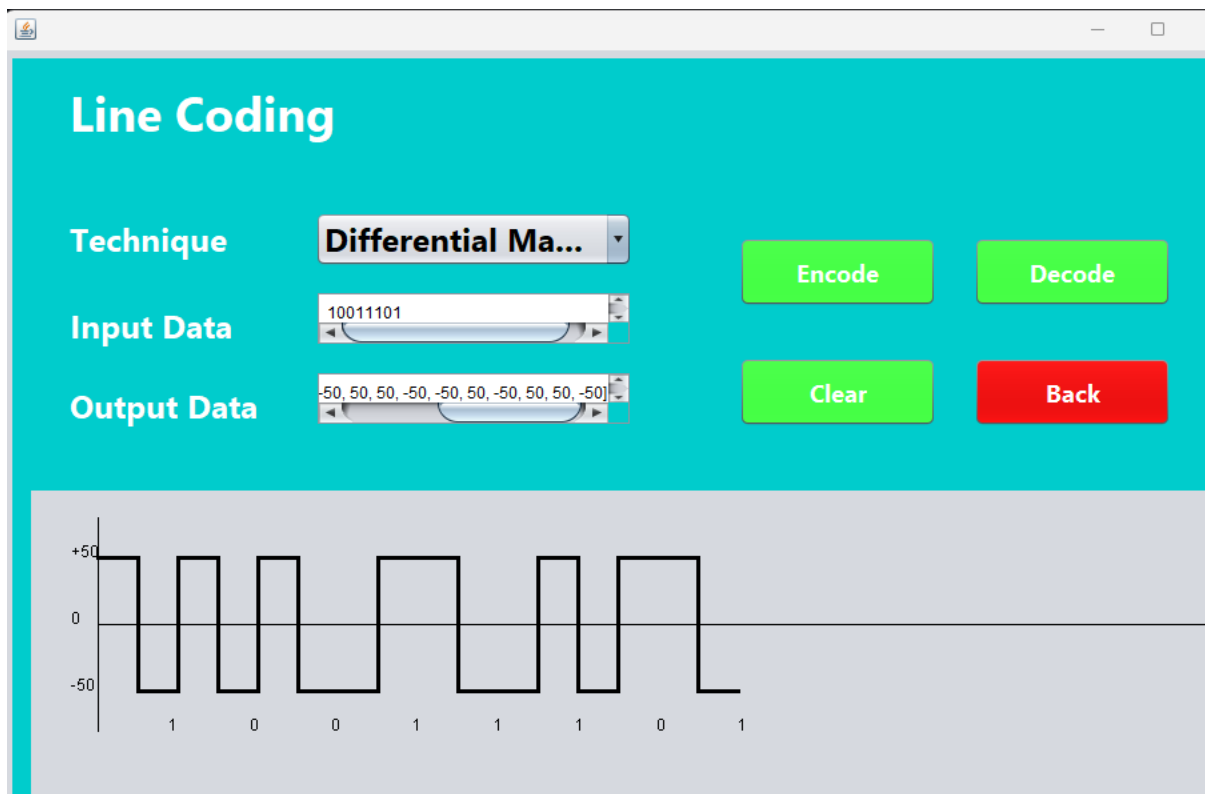


Fig: Differential Manchester

When users click on the Decode button, then they will see the Decoded bits of the differential Manchester graph.

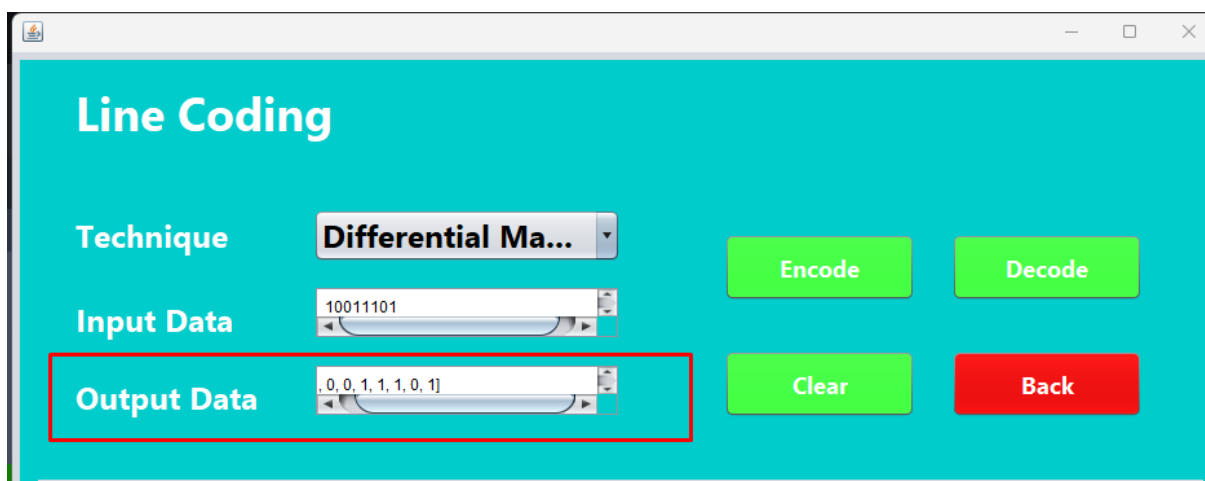


Fig: Differntial Manchester

2.2.5 AMI:

After Choosing the option AMI users can see the interface for AMI, After inserting the input they will get a graph of AMI Line Coding. After inserting the Input users click on the Encode button, then they will see the AMI Graph.

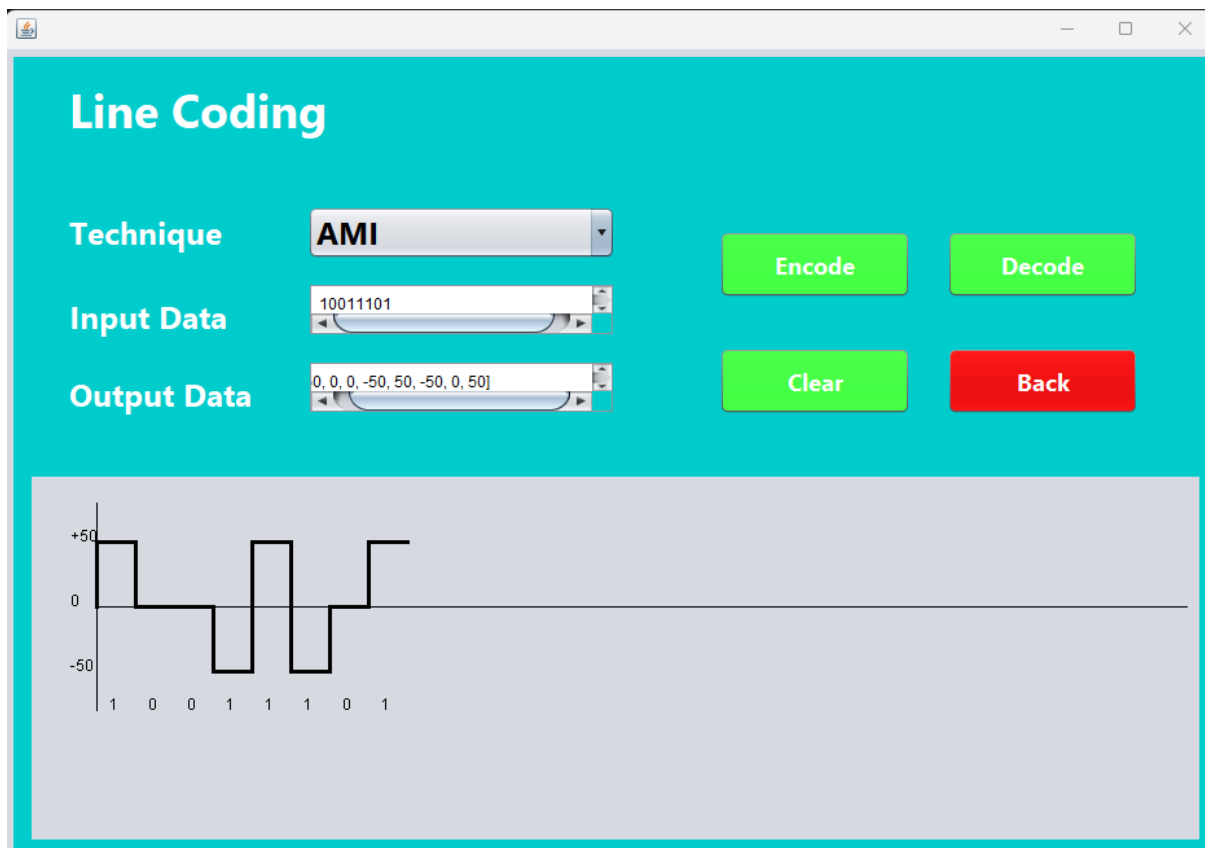


Fig: AMI Encoding

When users click on the Decode button, then they will see the Decoded bits of the AMI graph.

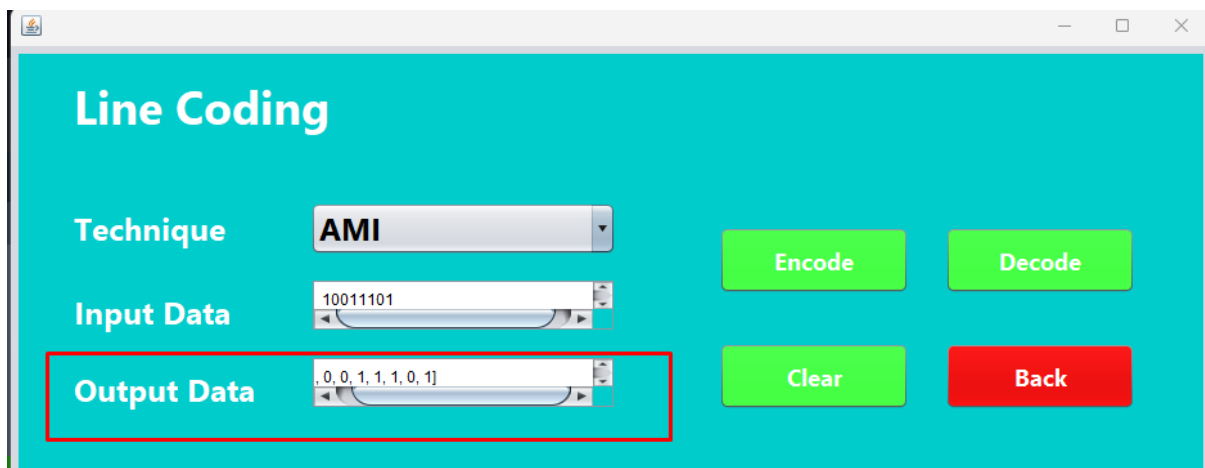


Fig: AMI Decoding

2.2.6 Pseudo Ternary:

After Choosing the option Pseudo Ternary users can see the interface for Pseudo Ternary, After inserting the input they will get a graph of Pseudo Ternary Line Coding. After inserting the Input users click on the Encode button, then they will see the Pseudo Ternary Graph.

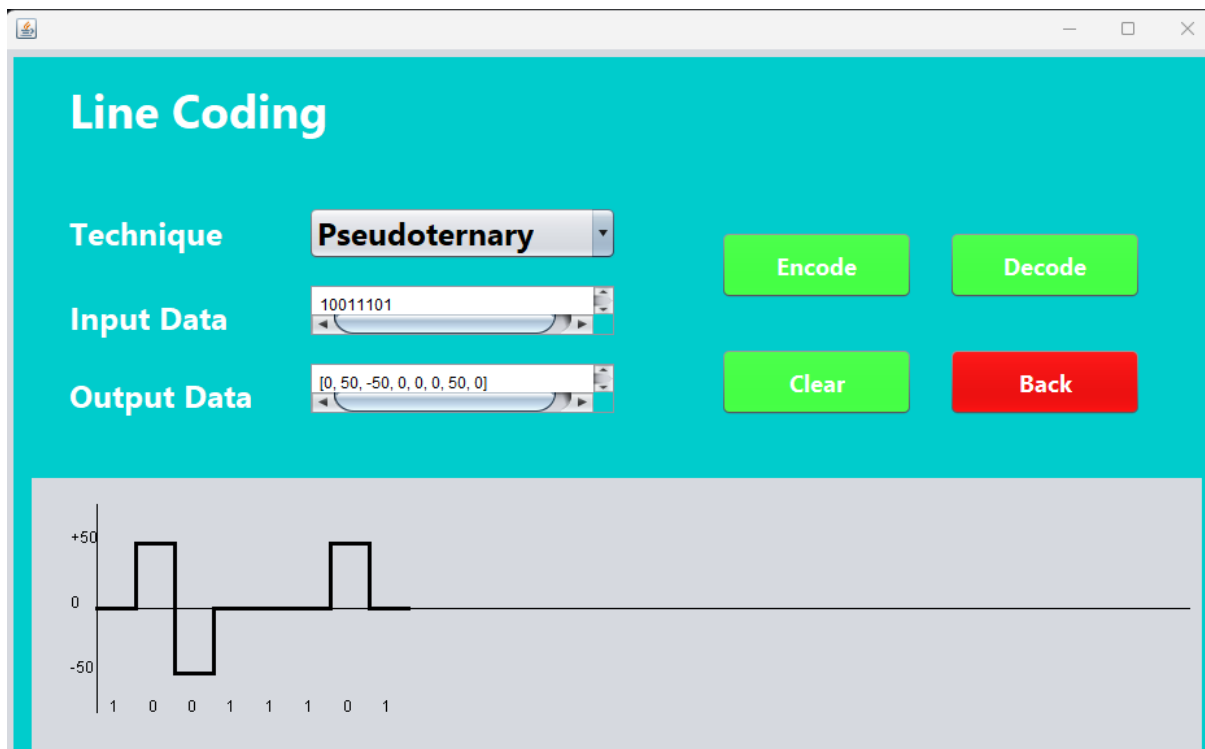


Fig: Pseudo Ternary Encoding

When users click on the Decode button, then they will see the Decoded bits of the Pseudo ternary graph.

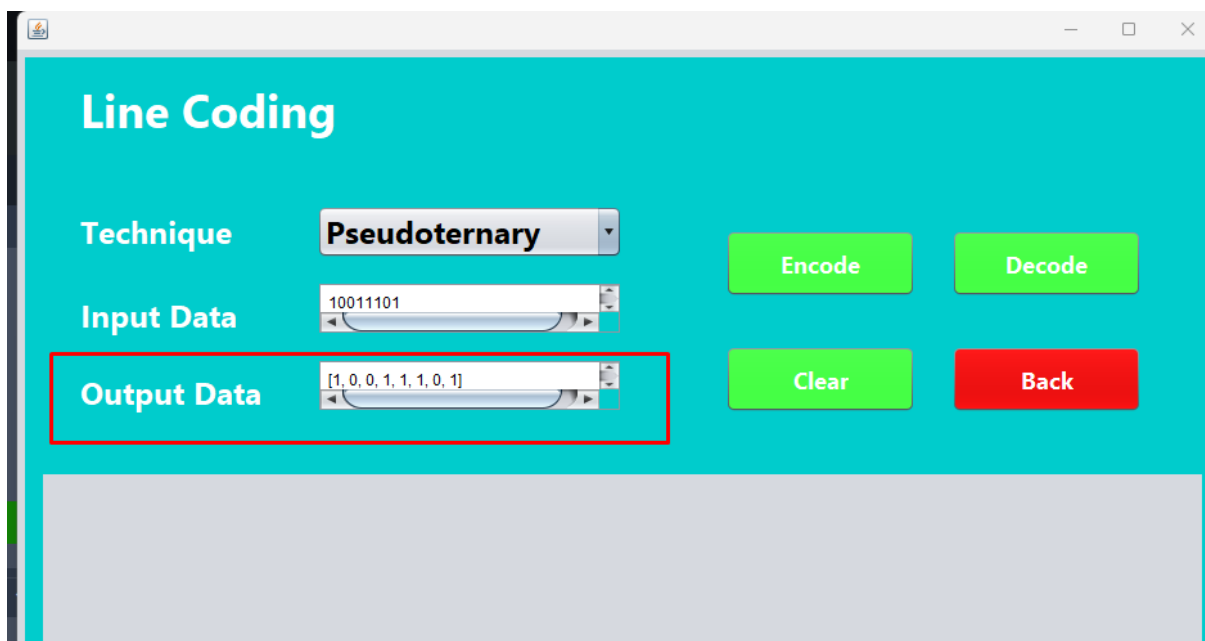


Fig: Pseudoternary Decoding

2.2.7 Exit option:

If the user wants to exit from the system, they need to back to the main menu and click on Logout Button.

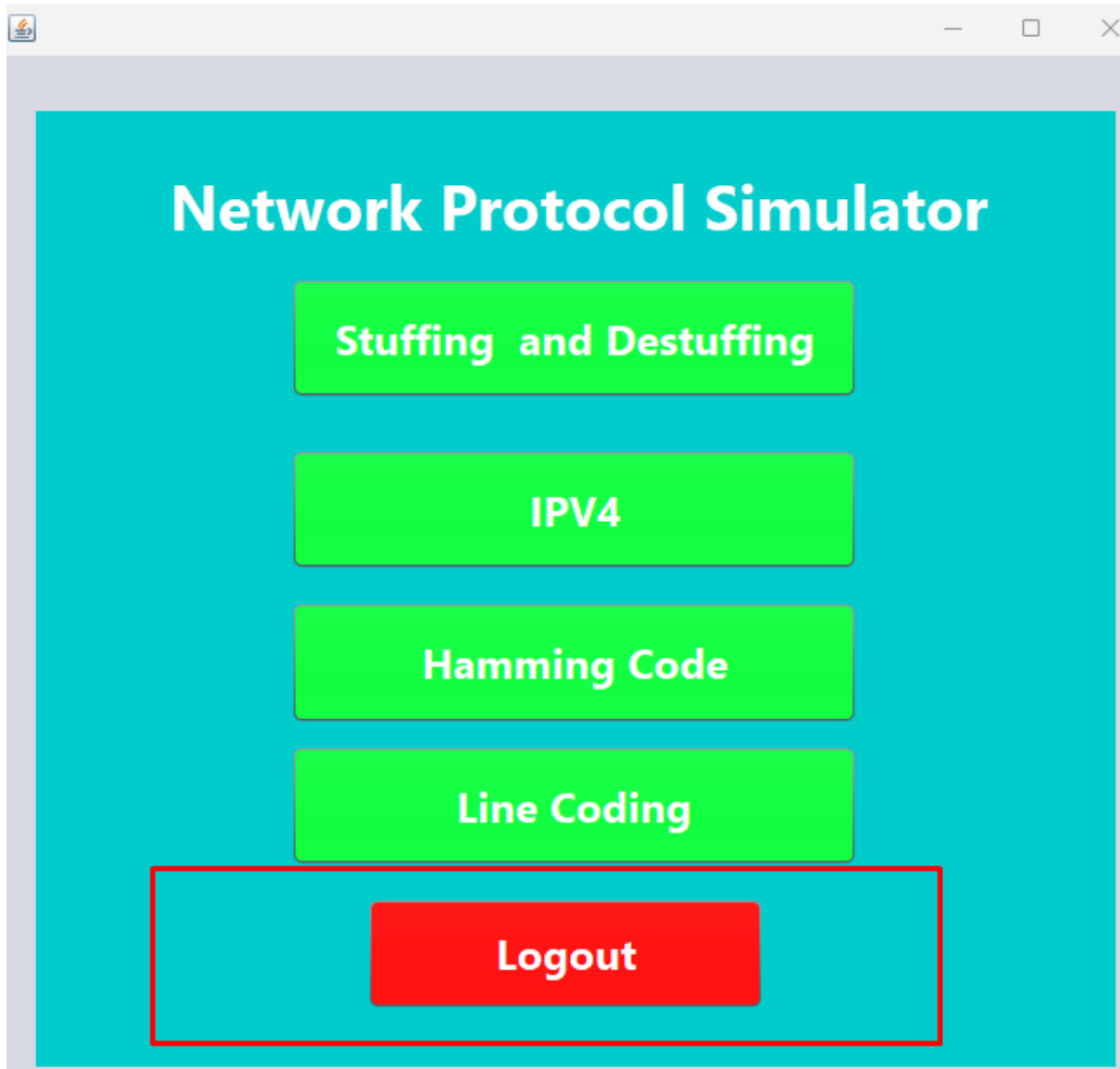


Fig: Logout